

当然，在 C 程序中包含汇编代码使得这些代码与某类特殊的机器相关(例如 x86-64)，所以只应该在想要的特性只能以此种方式才能访问到时才使用它。

3.3 数据格式

由于是从 16 位体系结构扩展成 32 位的，Intel 用术语“字(word)”表示 16 位数据类型。因此，称 32 位数为“双字(double words)”，称 64 位数为“四字(quad words)”。图 3-1 给出了 C 语言基本数据类型对应的 x86-64 表示。标准 int 值存储为双字(32 位)。指针(在此用 char * 表示)存储为 8 字节的四字，64 位机器本来就预期如此。x86-64 中，数据类型 long 实现为 64 位，允许表示的值范围较大。本章代码示例中的大部分都使用了指针和 long 数据类型，所以都是四字操作。x86-64 指令集同样包括完整的针对字节、字和双字的指令。

C 声明	Intel 数据类型	汇编代码后缀	大小(字节)
char	字节	b	1
short	字	w	2
int	双字	l	4
long	四字	q	8
char*	四字	q	8
float	单精度	s	4
double	双精度	l	8

图 3-1 C 语言数据类型在 x86-64 中的大小。在 64 位机器中，指针长 8 字节

浮点数主要有两种形式：单精度(4 字节)值，对应于 C 语言数据类型 float；双精度(8 字节)值，对应于 C 语言数据类型 double。x86 家族的微处理器历史上实现过对一种特殊的 80 位(10 字节)浮点格式进行全套的浮点运算(参见家庭作业 2.86)。可以在 C 程序中使用声明 long double 来指定这种格式。不过我们不建议使用这种格式。它不能移植到其他类型的机器上，而且实现的硬件也不如单精度和双精度算术运算的高效。

如图所示，大多数 GCC 生成的汇编代码指令都有一个字符的后缀，表明操作数的大小。例如，数据传送指令有四个变种：movb(传送字节)、movw(传送字)、movl(传送双字)和 movq(传送四字)。后缀‘l’用来表示双字，因为 32 位数被看成是“长字(long word)”。注意，汇编代码也使用后缀‘l’来表示 4 字节整数和 8 字节双精度浮点数。这不会产生歧义，因为浮点数使用的是一组完全不同的指令和寄存器。

3.4 访问信息

一个 x86-64 的中央处理单元(CPU)包含一组 16 个存储 64 位值的通用目的寄存器。这些寄存器用来存储整数数据和指针。图 3-2 显示了这 16 个寄存器。它们的名字都以 %r 开头，不过后面还跟着一些不同的命名规则的名字，这是由于指令集历史演化造成的。最初的 8086 中有 8 个 16 位的寄存器，即图 3-2 中的 %ax 到 %bp。每个寄存器都有特殊的用途，它们的名字就反映了这些不同的用途。扩展到 IA32 架构时，这些寄存器也扩展成 32 位寄存器，标号从 %eax 到 %ebp。扩展到 x86-64 后，原来的 8 个寄存器扩展成 64 位，标号从 %rax 到 %rbp。除此之外，还增加了 8 个新的寄存器，它们的标号是按照新的命名规则制定的：从 %r8 到 %r15。

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		被调用者保存
%rcx	%ecx	%cx	%cl		第4个参数
%rdx	%edx	%dx	%dl		第3个参数
%rsi	%esi	%si	%sil		第2个参数
%rdi	%edi	%di	%dil		第1个参数
%rbp	%ebp	%bp	%bpl		被调用者保存
%rsp	%esp	%sp	%spl		栈指针
%r8	%r8d	%r8w	%r8b		第5个参数
%r9	%r9d	%r9w	%r9b		第6个参数
%r10	%r10d	%r10w	%r10b		调用者保存
%r11	%r11d	%r11w	%r11b		调用者保存
%r12	%r12d	%r12w	%r12b		被调用者保存
%r13	%r13d	%r13w	%r13b		被调用者保存
%r14	%r14d	%r14w	%r14b		被调用者保存
%r15	%r15d	%r15w	%r15b		被调用者保存

图 3-2 整数寄存器。所有 16 个寄存器的低位部分都可以作为字节、字(16 位)、双字(32 位)和四字(64 位)数字来访问

如图 3-2 中嵌套的方框标明的, 指令可以对这 16 个寄存器的低位字节中存放的不同大小的数据进行操作。字节级操作可以访问最低的字节, 16 位操作可以访问最低的 2 个字节, 32 位操作可以访问最低的 4 个字节, 而 64 位操作可以访问整个寄存器。

在后面的章节中, 我们会展现很多指令, 复制和生成 1 字节、2 字节、4 字节和 8 字节值。当这些指令以寄存器作为目标时, 对于生成小于 8 字节结果的指令, 寄存器中剩下的字节会怎么样, 对此有两条规则: 生成 1 字节和 2 字节数字的指令会保持剩下的字节不变; 生成 4 字节数字的指令会把高位 4 个字节置为 0。后面这条规则是作为从 IA32 到 x86-64 的扩展的一部分而采用的。

就像图 3-2 右边的解释说明的那样, 在常见的程序里不同的寄存器扮演不同的角色。其中最特别的是栈指针 `%rsp`, 用来指明运行时栈的结束位置。有些程序会明确地读写这个寄存器。另外 15 个寄存器的用法更灵活。少量指令会使用某些特定的寄存器。更重要的

是，有一组标准的编程规范控制着如何使用寄存器来管理栈、传递函数参数、从函数的返回值，以及存储局部和临时数据。我们会在描述过程的实现时（特别是在 3.7 节中），讲述这些惯例。

3.4.1 操作数指示符

大多数指令有一个或多个操作数(operand)，指示出执行一个操作中要使用的源数据值，以及放置结果的目的位置。x86-64 支持多种操作数格式(参见图 3-3)。源数据值可以以常数形式给出，或是从寄存器或内存中读出。结果可以存放在寄存器或内存中。因此，各种不同的操作数的可能性被分为三种类型。第一种类型是立即数(immediate)，用来表示常数值。在 ATT 格式的汇编代码中，立即数的书写方式是‘\$’后面跟一个用标准 C 表示法表示的整数，比如，\$-577 或 \$0x1F。不同的指令允许的立即数值范围不同，汇编器会自动选择最紧凑的方式进行数值编码。第二种类型是寄存器(register)，它表示某个寄存器的内容，16 个寄存器的低位 1 字节、2 字节、4 字节或 8 字节中的一个作为操作数，这些字节数分别对应于 8 位、16 位、32 位或 64 位。在图 3-3 中，我们用符号 r_a 来表示任意寄存器 a ，用引用 $R[r_a]$ 来表示它的值，这是将寄存器集合看成一个数组 R ，用寄存器标识符作为索引。

第三类操作数是内存引用，它会根据计算出来的地址(通常称为有效地址)访问某个内存位置。因为将内存看成一个很大的字节数组，我们用符号 $M_b[Addr]$ 表示对存储在内存中从地址 $Addr$ 开始的 b 个字节值的引用。为了简便，我们通常省去下标 b 。

如图 3-3 所示，有多种不同的寻址模式，允许不同形式的内存引用。表中底部用语法 $Imm(r_b, r_i, s)$ 表示的是最常用的形式。这样的引用有四个组成部分：一个立即数偏移 Imm ，一个基址寄存器 r_b ，一个变址寄存器 r_i 和一个比例因子 s ，这里 s 必须是 1、2、4 或者 8。基址和变址寄存器都必须是 64 位寄存器。有效地址被计算为 $Imm + R[r_b] + R[r_i] \cdot s$ 。引用数组元素时，会用到这种通用形式。其他形式都是这种通用形式的特殊情况，只是省略了某些部分。正如我们将看到的，当引用数组和结构元素时，比较复杂的寻址模式是很有用的。

类型	格式	操作数值	名称
立即数	$\$Imm$	Imm	立即数寻址
寄存器	r_a	$R[r_a]$	寄存器寻址
存储器	Imm	$M[Imm]$	绝对寻址
存储器	(r_a)	$M[R[r_a]]$	间接寻址
存储器	$Imm(r_b)$	$M[Imm + R[r_b]]$	(基址 + 偏移量) 寻址
存储器	(r_b, r_i)	$M[R[r_b] + R[r_i]]$	变址寻址
存储器	$Imm(r_b, r_i)$	$M[Imm + R[r_b] + R[r_i]]$	变址寻址
存储器	(r_i, s)	$M[R[r_i] \cdot s]$	比例变址寻址
存储器	$Imm(r_i, s)$	$M[Imm + R[r_i] \cdot s]$	比例变址寻址
存储器	(r_b, r_i, s)	$M[R[r_b] + R[r_i] \cdot s]$	比例变址寻址
存储器	$Imm(r_b, r_i, s)$	$M[Imm + R[r_b] + R[r_i] \cdot s]$	比例变址寻址

图 3-3 操作数格式。操作数可以表示立即数(常数)值、寄存器值或是来自内存的值。比例因子 s 必须是 1、2、4 或者 8

 练习题 3.1 假设下面的值存放在指明的内存地址和寄存器中：

地址	值	寄存器	值
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

填写下表，给出所示操作数的值：

操作数	值
%rax	
0x104	
\$0x108	
(%rax)	
4(%rax)	
9(%rax,%rdx)	
260(%rcx,%rdx)	
0xFC(,%rcx,4)	
(%rax,%rdx,4)	

3.4.2 数据传送指令

最频繁使用的指令是将数据从一个位置复制到另一个位置的指令。操作数表示的通用性使得一条简单的数据传送指令能够完成在许多机器中要好几条不同指令才能完成的功能。我们会介绍多种不同的数据传送指令，它们或者源和目的类型不同，或者执行的转换不同，或者具有的一些副作用不同。在我们的讲述中，把许多不同的指令划分成指令类，每一类中的指令执行相同的操作，只不过操作数大小不同。

图 3-4 列出的是最简单形式的的数据传送指令——MOV 类。这些指令把数据从源位置复制到目的位置，不做任何变化。MOV 类由四条指令组成：movb、movw、movl 和 movq。这些指令都执行同样的操作；主要区别在于它们操作的数据大小不同：分别是 1、2、4 和 8 字节。

指令	效果	描述
MOV S, D	D←S	传送
movb		传送字节
movw		传送字
movl		传送双字
movq		传送四字
movabsq I, R	R←I	传送绝对的四字

图 3-4 简单的数据传送指令

源操作数指定的值是一个立即数，存储在寄存器中或者内存中。目的操作数指定一个位置，要么是一个寄存器或者，要么是一个内存地址。x86-64 加了一条限制，传送指令的两个操作数不能都指向内存位置。将一个值从一个内存位置复制到另一个内存位置需要两条指令——第一条指令将源值加载到寄存器中，第二条将该寄存器值写入目的位置。参考图 3-2，这些指令的寄存器操作数可以是 16 个寄存器有标号部分中的任意一个，寄存器部

分的大小必须与指令最后一个字符(‘b’, ‘w’, ‘l’或‘q’)指定的大小匹配。大多数情况中, MOV 指令只会更新目的操作数指定的那些寄存器字节或内存位置。唯一的例外是 movl 指令以寄存器作为目的时, 它会把该寄存器的高位 4 字节设置为 0。造成这个例外的原因是 x86-64 采用的惯例, 即任何为寄存器生成 32 位值的指令都会把该寄存器的高位部分置成 0。

下面的 MOV 指令示例给出了源和目的类型的五种可能的组合。记住, 第一个是源操作数, 第二个是目的操作数:

```

1    movl $0x4050,%eax      Immediate--Register, 4 bytes
2    movw %bp,%sp           Register--Register, 2 bytes
3    movb (%rdi,%rcx),%al    Memory--Register, 1 byte
4    movb $-17,(%rsp)        Immediate--Memory, 1 byte
5    movq %rax,-12(%rbp)     Register--Memory, 8 bytes

```

图 3-4 中记录的最后一条指令是处理 64 位立即数数据的。常规的 movq 指令只能以表示为 32 位补码数字的立即数作为源操作数, 然后把这个值符号扩展得到 64 位的值, 放到目的位置。movabsq 指令能够以任意 64 位立即数值作为源操作数, 并且只能以寄存器作为目的。

图 3-5 和图 3-6 记录的是两类数据移动指令, 在将较小的源值复制到较大的目的时使用。所有这些指令都把数据从源(在寄存器或内存中)复制到目的寄存器。MOVZ 类中的指令把目的中剩余的字节填充为 0, 而 MOVS 类中的指令通过符号扩展来填充, 把源操作的最高位进行复制。可以观察到, 每条指令名字的最后两个字符都是大小指示符: 第一个字符指定源的大小, 而第二个指明目的的大小。正如看到的那样, 这两个类中每个都有三条指令, 包括了所有的源大小为 1 个和 2 个字节、目的的大小为 2 个和 4 个的情况, 当然只考虑目的大于源的情况。

指令	效果	描述
MOVZ S, R	R ← 零扩展(S)	以零扩展进行传送
movzbw		将做了零扩展的字节传送到字
movzbl		将做了零扩展的字节传送到双字
movzwl		将做了零扩展的字传送到双字
movzbq		将做了零扩展的字节传送到四字
movzwb		将做了零扩展的字传送到四字

图 3-5 零扩展数据传送指令。这些指令以寄存器或内存地址作为源, 以寄存器作为目的

指令	效果	描述
MOVS S, R	R ← 符号扩展(S)	传送符号扩展的字节
movsbw		将做了符号扩展的字节传送到字
movsbl		将做了符号扩展的字节传送到双字
movswl		将做了符号扩展的字传送到双字
movsbq		将做了符号扩展的字节传送到四字
movswq		将做了符号扩展的字传送到四字
movslq		将做了符号扩展的双字传送到四字
cltq	%rax ← 符号扩展(%eax)	把 %eax 符号扩展到 %rax

图 3-6 符号扩展数据传送指令。MOVS 指令以寄存器或内存地址作为源, 以寄存器作为目的。cltq 指令只作用于寄存器 %eax 和 %rax

旁注 理解数据传送如何改变目的寄存器

正如我们描述的那样，关于数据传送指令是否以及如何修改目的寄存器的高位字节有两种不同的方法。下面这段代码序列会说明其差别：

```

1      movabsq $0x0011223344556677, %rax    %rax = 0011223344556677
2      movb    $-1, %al                     %rax = 00112233445566FF
3      movw    $-1, %ax                     %rax = 001122334455FFFF
4      movl    $-1, %eax                    %rax = 00000000FFFFFFFF
5      movq    $-1, %rax                    %rax = FFFFFFFFFFFFFFFF

```

在接下来的讨论中，我们使用十六进制表示。在这个例子中，第 1 行的指令把寄存器 %rax 初始化为位模式 0011223344556677。剩下的指令的源操作数值是立即数值 -1。回想 -1 的十六进制表示形如 FF...F，这里 F 的数量是表述中字节数量的两倍。因此 movb 指令（第 2 行）把 %rax 的低位字节设置为 FF，而 movw 指令（第 3 行）把低 2 字节设置为 FFFF，剩下的字节保持不变。movl 指令（第 4 行）将低 4 个字节设置为 FFFFFFFF，同时把高位 4 字节设置为 00000000。最后 movq 指令（第 5 行）把整个寄存器设置为 FFFFFFFFFFFFFFFF。

注意图 3-5 中并没有一条明确的指令把 4 字节源值零扩展到 8 字节目的。这样的指令逻辑上应该被命名为 movzsq，但是并没有这样的指令。不过，这样的数据传送可以用以寄存器为目的的 movl 指令来实现。这一技术利用的属性是，生成 4 字节值并以寄存器作为目的的指令会把高 4 字节置为 0。对于 64 位的目标，所有三种源类型都有对应的符号扩展传送，而只有两种较小的源类型有零扩展传送。

图 3-6 还给出 cltq 指令。这条指令没有操作数：它总是以寄存器 %eax 作为源，%rax 作为符号扩展结果的目的。它的效果与指令 movslq %eax, %rax 完全一致，不过编码更紧凑。

 **练习题 3.2** 对于下面汇编代码的每一行，根据操作数，确定适当的指令后缀。（例如，mov 可以被重写成 movb、movw、movl 或者 movq。）

```

mov__    %eax, (%rsp)
mov__    (%rax), %dx
mov__    $0xFF, %bl
mov__    (%rsp,%rdx,4), %dl
mov__    (%rdx), %rax
mov__    %dx, (%rax)

```

旁注 字节传送指令比较

下面这个示例说明了不同的数据传送指令如何改变或者不改变目的的高位字节。仔细观察可以发现，三个字节传送指令 movb、movsbq 和 movzbq 之间有细微的差别。示例如下：

```

1      movabsq $0x0011223344556677, %rax    %rax = 0011223344556677
2      movb    $0xAA, %dl                   %dl  = AA
3      movb    %dl, %al                     %rax = 00112233445566AA
4      movsbq  %dl, %rax                    %rax = FFFFFFFF000000AA
5      movzbq  %dl, %rax                    %rax = 00000000000000AA

```

在下面的讨论中，所有的值都使用十六进制表示。代码的头 2 行将寄存器 %rax 和 %dl 分别初始化为 0011223344556677 和 AA。剩下的指令都是将 %rdx 的低位字节复制到 %rax 的低位字节。movb 指令（第 3 行）不改变其他字节。根据源字节的最高位，movsbq 指令（第 4 行）将其他 7 个字节设为全 1 或全 0。由于十六进制 A 表示二进制值 1010，符号扩展会把高位字节都设置为 FF。movzbq 指令（第 5 行）总是将其他 7 个字节全都设置为 0。

 **练习题 3.3** 当我们调用汇编器的时候，下面代码的每一行都会产生一个错误消息。解释每一行都是哪里出了错。

```
movb $0xF, (%ebx)
movl %rax, (%rsp)
movw (%rax), 4(%rsp)
movb %al, %sl
movq %rax, $0x123
movl %eax, %rdx
movb %si, 8(%rbp)
```

3.4.3 数据传送示例

作为一个使用数据传送指令的代码示例，考虑图 3-7 中所示的数据交换函数，既有 C 代码，也有 GCC 产生的汇编代码。

```
long exchange(long *xp, long y)
{
    long x = *xp;
    *xp = y;
    return x;
}
```

a) C语言代码

```
long exchange(long *xp, long y)
xp in %rdi, y in %rsi
1  exchange:
2      movq    (%rdi), %rax    Get x at xp. Set as return value.
3      movq    %rsi, (%rdi)    Store y at xp.
4      ret                                Return.
```

b) 汇编代码

图 3-7 exchange 函数的 C 语言和汇编代码。寄存器 %rdi 和 %rsi 分别存放参数 xp 和 y

如图 3-7b 所示，函数 exchange 由三条指令实现：两个数据传送(movq)，加上一条返回函数被调用点的指令(ret)。我们会在 3.7 节中讲述函数调用和返回的细节。在此之前，知道参数通过寄存器传递给函数就足够了。我们对汇编代码添加注释来加以说明。函数通过把值存储在寄存器 %rax 或该寄存器的某个低位部分中返回。

当过程开始执行时，过程参数 xp 和 y 分别存储在寄存器 %rdi 和 %rsi 中。然后，指令 2 从内存中读出 x，把它存放到寄存器 %rax 中，直接实现了 C 程序中的操作 $x = *xp$ 。稍后，用寄存器 %rax 从这个函数返回一个值，因而返回值就是 x。指令 3 将 y 写入到寄存器 %rdi 中的 xp 指向的内存位置，直接实现了操作 $*xp = y$ 。这个例子说明了如何用 MOV 指令从内存中读值到寄存器(第 2 行)，如何从寄存器写到内存(第 3 行)。

关于这段汇编代码有两点值得注意。首先，我们看到 C 语言中所谓的“指针”其实就是地址。间接引用指针就是将该指针放在一个寄存器中，然后在内存引用中使用这个寄存器。其次，像 x 这样的局部变量通常是保存在寄存器中，而不是内存中。访问寄存器比访问内存要快得多。

 练习题 3.4 假设变量 `sp` 和 `dp` 被声明为类型

```
src_t *sp;
dest_t *dp;
```

这里 `src_t` 和 `dest_t` 是用 `typedef` 声明的数据类型。我们想使用适当的数据传送指令来实现下面的操作

```
*dp = (dest_t) *sp;
```

假设 `sp` 和 `dp` 的值分别存储在寄存器 `%rdi` 和 `%rsi` 中。对于表中的每个表项，给出实现指定数据传送的两条指令。其中第一条指令应该从内存中读数，做适当的转换，并设置寄存器 `%rax` 的适当部分。然后，第二条指令要把 `%rax` 的适当部分写到内存。在这两种情况中，寄存器的部分可以是 `%rax`、`%eax`、`%ax` 或 `%al`，两者可以互不相同。

记住，当执行强制类型转换既涉及大小变化又涉及 C 语言中符号变化时，操作应该先改变大小(2.2.6 节)。

src_t	dest_t	指令
long	long	movq(%rdi),%rax movq %rax, (%rsi)
char	int	_____
char	unsigned	_____
unsigned char	long	_____
int	char	_____
unsigned	unsigned char	_____
char	short	_____

给 C 语言初学者 指针的一些示例

函数 `exchange`(图 3-7a)提供了一个关于 C 语言中指针使用的很好说明。参数 `xp` 是一个指向 `long` 类型的整数的指针，而 `y` 是一个 `long` 类型的整数。语句

```
long x = *xp;
```

表示我们将读存储在 `xp` 所指位置中的值，并将它存放到名字为 `x` 的局部变量中。这个读操作称为指针的间接引用(pointer dereferencing)，C 操作符 `*` 执行指针的间接引用。

语句

```
*xp = y;
```

正好相反——它将参数 `y` 的值写到 `xp` 所指的位置。这也是指针间接引用的一种形式(所有操作符 `*`)，但是它表明的是一个写操作，因为它在赋值语句的左边。

下面是调用 `exchange` 的一个实际例子：

```
long a = 4;
long b = exchange(&a, 3);
printf("a = %ld, b = %ld\n", a, b);
```


这段代码会打印出：

```
a = 3, b = 4
```

C 操作符 `&` (称为“取址”操作符) 创建一个指针，在本例中，该指针指向保存局部变量 `a` 的位置。然后，函数 `exchange` 将用 3 覆盖存储在 `a` 中的值，但是返回原来的值 4 作为函数的值。注意如何将指针传递给 `exchange`，它能修改存在某个远处位置的数据。

 练习题 3.5 已知信息如下。将一个原型为

```
void decode1(long *xp, long *yp, long *zp);
```

的函数编译成汇编代码，得到如下代码：

```
void decode1(long *xp, long *yp, long *zp)
  xp in %rdi, yp in %rsi, zp in %rdx
decode1:
  movq    (%rdi), %r8
  movq    (%rsi), %rcx
  movq    (%rdx), %rax
  movq    %r8, (%rsi)
  movq    %rcx, (%rdx)
  movq    %rax, (%rdi)
  ret
```

参数 `xp`、`yp` 和 `zp` 分别存储在对应的寄存器 `%rdi`、`%rsi` 和 `%rdx` 中。

请写出等效于上面汇编代码的 `decode1` 的 C 代码。

3.4.4 压入和弹出栈数据

最后两个数据传送操作可以将数据压入程序栈中，以及从程序栈中弹出数据，如图 3-8 所示。正如我们将看到的，栈在处理过程调用中起到至关重要的作用。栈是一种数据结构，可以添加或者删除值，不过要遵循“后进先出”的原则。通过 `push` 操作把数据压入栈中，通过 `pop` 操作删除数据；它具有一个属性：弹出的值永远是最近被压入而且仍然在栈中的值。栈可以实现为一个数组，总是从数组的一端插入和删除元素。这一端被称为栈顶。在 x86-64 中，程序栈存放在内存中某个区域。如图 3-9 所示，栈向下增长，这样一来，栈顶元素的地址是所有栈中元素地址中最低的。（根据惯例，我们的栈是倒过来画的，栈“顶”在图的底部。）栈指针 `%rsp` 保存着栈顶元素的地址。

指令	效果	描述
<code>pushq S</code>	$R[\%rsp] \leftarrow R[\%rsp] - 8;$ $M[R[\%rsp]] \leftarrow S$	将四字压入栈
<code>popq D</code>	$D \leftarrow M[R[\%rsp]];$ $R[\%rsp] \leftarrow R[\%rsp] + 8$	将四字弹出栈

图 3-8 入栈和出栈指令

`pushq` 指令的功能是把数据压入到栈上，而 `popq` 指令是弹出数据。这些指令都只有一个操作数——压入的数据源和弹出的数据目的。

将一个四字值压入栈中，首先要将栈指针减 8，然后将值写到新的栈顶地址。因此，指令 `pushq %rbp` 的行为等价于下面两条指令：

```
subq $8,%rsp      Decrement stack pointer
movq %rbp, (%rsp) Store %rbp on stack
```

它们之间的区别是在机器代码中 pushq 指令编码为 1 个字节，而上面那两条指令一共需要 8 个字节。图 3-9 中前两栏给出的是，当 %rsp 为 0x108, %rax 为 0x123 时，执行指令 pushq %rax 的效果。首先 %rsp 会减 8，得到 0x100，然后会将 0x123 存放到内存地址 0x100 处。

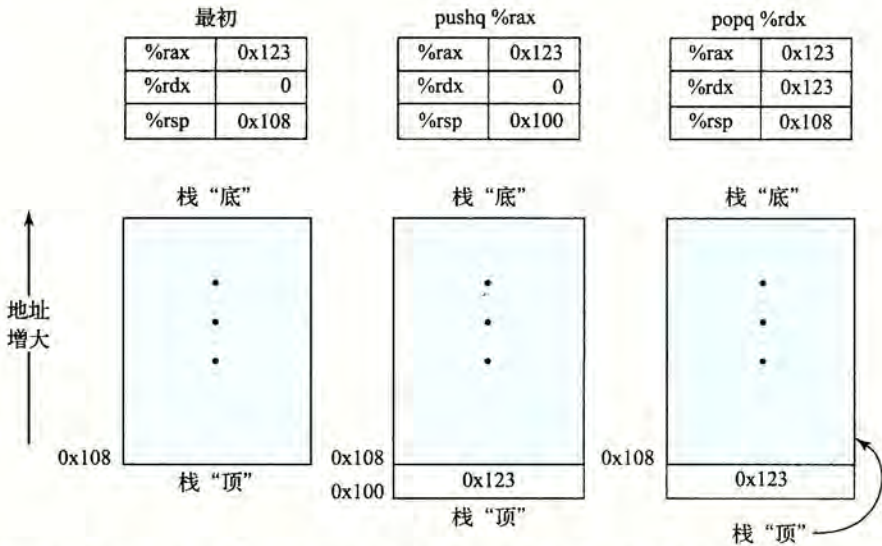


图 3-9 栈操作说明。根据惯例，我们的栈是倒过来画的，因而栈“顶”在底部。x86-64 中，栈向低地址方向增长，所以压栈是减小栈指针（寄存器 %rsp）的值，并将数据存放到内存中，而出栈是从内存中读数据，并增加栈指针的值

弹出一个四字的操作包括从栈顶位置读出数据，然后将栈指针加 8。因此，指令 popq %rax 等价于下面两条指令：

```
movq (%rsp), %rax      Read %rax from stack
addq $8, %rsp          Increment stack pointer
```

图 3-9 的第三栏说明的是在执行完 pushq 后立即执行指令 popq %rdx 的效果。先从内存中读出值 0x123，再写到寄存器 %rdx 中，然后，寄存器 %rsp 的值将增加回到 0x108。如图中所示，值 0x123 仍然会保持在内存位置 0x100 中，直到被覆盖（例如被另一条入栈操作覆盖）。无论如何，%rsp 指向的地址总是栈顶。

因为栈和程序代码以及其他形式的程序数据都是放在同一内存中，所以程序可以用标准的内存寻址方法访问栈内的任意位置。例如，假设栈顶元素是四字，指令 movq 8(%rsp), %rdx 会将第二个四字从栈中复制到寄存器 %rdx。

3.5 算术和逻辑操作

图 3-10 列出了 x86-64 的一些整数和逻辑操作。大多数操作都分成了指令类，这些指令类有各种带不同大小操作数的变种（只有 leaq 没有其他大小的变种）。例如，指令类 ADD 由四条加法指令组成：addb、addw、addl 和 addq，分别是字节加法、字加法、双字加法和四字加法。事实上，给出的每个指令类都有对这四种不同大小数据的指令。这些